MANJARES DEL CAMPO

Documentación Técnica Completa

Plataforma de E-Commerce de Productos del Campo

Versión: 1.0
Fecha: 2026

Repositorio: github.com/emilymontec/MANJARES-CAMPO

1. Introducción

MANJARES DEL CAMPO es una plataforma de comercio electrónico (e-commerce) fullstack diseñada para la venta de productos agrícolas y artesanales del campo. Conecta a productores locales con compradores finales a través de una interfaz moderna, intuitiva y funcional.

La plataforma gestiona el ciclo completo de compra: desde la navegación del catálogo hasta la confirmación del pago, el rastreo del envío y las notificaciones al cliente vía WhatsApp.

1.1 Objetivos del Sistema

- **Objetivo General**: Proveer una plataforma digital que facilite la comercialización de productos del campo de forma directa y eficiente.
- **Accesibilidad**: Permitir que compradores accedan al catálogo y realicen pedidos desde cualquier dispositivo.
- **Comunicación**: Mantener al cliente informado en tiempo real sobre el estado de su pedido mediante WhatsApp.
- **Administración**: Ofrecer al negocio un panel centralizado para gestionar productos, pedidos y envíos.

1.2 Alcance

El sistema cubre los siguientes procesos de negocio:

- Gestión del catálogo de productos del campo
- Carrito de compras y proceso de pago
- Confirmación y gestión de pedidos
- Rastreo de envíos.
- Notificaciones automáticas al cliente por WhatsApp
- Panel de administración para el operador del negocio

2. Stack Tecnológico

Capa	Tecnología	Descripción
Backend	Python 3.12 + Django	Framework web principal, ORM, autenticación, admin
Base de Datos	PostgreSQL	RDBMS principal para persistencia de datos
Frontend	HTML5 + CSS3 + JavaScript	Templates Django (server-side rendering)
Integración	Plugin WhatsApp	Notificaciones y confirmaciones al cliente
ORM	Django ORM	Abstracción de base de datos y migraciones
Autenticación	Django Auth	Gestión de sesiones y usuarios
Admin	Django Admin	Panel de administración integrado y extendido

2.1 Requisitos del Sistema

Requisito	Versión/Detalle
Python	3.12 o superior
Django	6.0
PostgreSQL	13 o superior
Node.js (opcional)	Para assets frontend
Sistema Operativo	Linux/macOS/Windows
Navegadores Soportados	Chrome, Firefox, Safari, Edge (modernos)

3. Arquitectura del Sistema

MANJARES DEL CAMPO sigue el patrón de arquitectura MVT (Model-View-Template) de Django, con separación clara de responsabilidades entre la lógica de negocio, la gestión de datos y la presentación.

3.1 Patrón MVT (Model-View-Template)

Componente	Responsabilidad	Ejemplos en el Proyecto
Model	Define la estructura de datos y lógica de negocio	Producto, Pedido, Cliente, Envio
View	Procesa requests HTTP y devuelve responses	CatalogoView, CarritoView, PedidoView
Template	Renderiza HTML con datos del contexto	catalogo.html, carrito.html, pedido.html

3.2 Flujo de una Solicitud

El ciclo de vida de una petición HTTP en la plataforma sigue el siguiente flujo:

```
Cliente (Navegador)
    │
    ▼
URLs (urls.py)  →  Despacha la ruta al View correspondiente
    │
    ▼
View (views.py)  →  Procesa lógica, consulta Models
    │
    ▼
Model (models.py) →  Interactúa con PostgreSQL
    │
    ▼
Template (.html)  →  Renderiza HTML con contexto
    │
    ▼
Respuesta HTTP al Cliente
```

3.3 Estructura de Carpetas del Proyecto

```
MANJARES-CAMPO/
├── manage.py                  # CLI de Django
├── requirements.txt           # Dependencias Python
├── .env                       # Variables de entorno (no versionado)
├── manjares/                  # Configuración principal del proyecto
│   ├── settings.py            # Configuración Django
│   ├── urls.py                # URLs raíz
│   ├── wsgi.py                # Servidor WSGI
│   └── asgi.py                # Servidor ASGI
├── catalogo/                  # App: catálogo de productos
│   ├── models.py              # Modelos: Producto, Categoria
│   ├── views.py               # Vistas del catálogo
│   ├── urls.py                # Rutas de catalogo
│   └── templates/catalogo/    # Templates HTML
├── carrito/                   # App: carrito de compras
│   ├── models.py              # Modelo: ItemCarrito
│   ├── views.py               # Lógica del carrito
│   └── templates/carrito/
├── pedidos/                   # App: gestión de pedidos
│   ├── models.py              # Modelos: Pedido, DetallePedido
│   ├── views.py               # Vistas de pedidos
│   ├── admin.py               # Registro en panel admin
│   └── templates/pedidos/
├── envios/                    # App: rastreo de envíos
│   ├── models.py              # Modelo: Envio, EstadoEnvio
│   └── views.py
├── whatsapp/                  # App: integración WhatsApp
│   └── utils.py               # Funciones de envío de mensajes
└── static/                    # Archivos estáticos (CSS, JS, imágenes)
```

4. Modelos de Datos

A continuación se describe el modelo de datos principal del sistema y las relaciones entre entidades.

4.1 Entidad: Producto

Campo	Tipo	Descripción
id	AutoField (PK)	Identificador único del producto
nombre	CharField(200)	Nombre del producto
descripcion	TextField	Descripción detallada
precio	DecimalField(10,2)	Precio unitario en moneda local
stock	PositiveIntegerField	Unidades disponibles en inventario
categoria	ForeignKey → Categoria	Categoría a la que pertenece
imagen	ImageField	Foto del producto
activo	BooleanField	Si el producto está visible en el catálogo
fecha_creacion	DateTimeField (auto)	Fecha de registro del producto

4.2 Entidad: Pedido

Campo	Tipo	Descripción
id	AutoField (PK)	Identificador único del pedido
cliente_nombre	CharField(200)	Nombre completo del cliente
cliente_telefono	CharField(20)	Número de WhatsApp del cliente
cliente_direccion	TextField	Dirección de entrega
estado	CharField (choices)	pendiente/confirmado/en_camino/entregado
total	DecimalField(10,2)	Monto total del pedido
pago_confirmado	BooleanField	Si el pago fue verificado por el administrador
fecha_pedido	DateTimeField (auto)	Fecha y hora en que se realizó el pedido

4.3 Entidad: Envío

Campo	Tipo	Descripción
id	AutoField (PK)	Identificador único del envío
pedido	OneToOneField → Pedido	Pedido asociado al envío
codigo_rastreo	CharField(100)	Código de seguimiento del paquete
estado_envio	CharField (choices)	preparando / despachado / en_ruta / entregado
transportadora	CharField(100)	Empresa de transporte
fecha_despacho	DateField	Fecha en que salió el paquete
fecha_entrega_est.	DateField	Fecha estimada de llegada al cliente

4.4 Diagrama de Relaciones

```
Categoria (1) ──────< Producto (N)
                          │
                          │ (a través de DetallePedido)
                          ▼
Cliente (1) ──────< Pedido (N) >────── DetallePedido
                          │
                          │ (OneToOne)
                          ▼
                        Envio (1)
```

5. Funcionalidades del Sistema

5.1 Catálogo de Productos

El módulo de catálogo es la vitrina principal de la tienda. Permite al cliente:

- Explorar todos los productos disponibles del campo
- Filtrar por categorías (frutas, verduras, lácteos, etc.)
- Ver disponibilidad de stock en tiempo real
- Agregar productos directamente al carrito desde el listado

Desde el panel de administración, el operador puede activar/desactivar productos, actualizar precios y gestionar el inventario.

5.2 Carrito de Compras

El carrito permite al cliente gestionar su selección antes de confirmar el pedido:

- Agregar uno o varios productos al carrito
- Modificar cantidades de cada ítem
- Eliminar productos del carrito
- Ver subtotal, total y resumen de la compra
- Continuar con el proceso de pago o seguir comprando

5.3 Confirmación de Pagos

El flujo de pago de la plataforma funciona de la siguiente manera:

1. El cliente realiza el pago por transferencia, consignación o medio acordado.
2. El cliente registra su pedido en la plataforma con sus datos de contacto y dirección.
3. El administrador revisa el pedido en el panel y confirma el pago manualmente.
4. Al confirmar el pago, el sistema actualiza el estado del pedido automáticamente.
5. Se envía notificación de confirmación al cliente vía WhatsApp.

5.4 Gestión de Pedidos

El módulo de pedidos centraliza toda la información de las órdenes de compra:

- Listado de todos los pedidos con filtros por ID.
- Vista detallada de cada pedido: productos, cantidades, total y datos del cliente
- Cambio de estado del pedido: pendiente → confirmado → en camino → entregado
- Confirmación manual de pagos por el administrador

5.5 Rastreo de Envíos

Una vez confirmado el pedido, se genera un envío con la siguiente información:

- Código de rastreo del paquete
- Estado actual del envío (preparando, despachado, en ruta, entregado)
- Cada actualización de estado genera una notificación automática al cliente por WhatsApp

5.6 Integración con WhatsApp

El plugin de WhatsApp es un componente clave del sistema de comunicación con el cliente. Se activa automáticamente en los siguientes eventos:

Evento Disparador	Mensaje Enviado al Cliente
Nuevo pedido	Confirmación de recepción del pedido con resumen
Pedido confirmado	Notificación de que el pago fue verificado
En preparación	Código de rastreo y transportadora asignada
Pedido enviado	Actualización de ubicación o estado del paquete
Pedido entregado	Confirmación de entrega y mensaje de satisfacción
Pedido cancelado	Pedido cancelado por el cliente

5.7 Panel de Administración

El panel de administración permite al operador del negocio:

- Gestionar el catálogo completo de productos
- Ver, filtrar y exportar pedidos
- Confirmar pagos recibidos
- Actualizar estados de pedidos y envíos
- Gestionar categorías de productos
- Ver métricas básicas: total de pedidos, ventas del día, productos más vendidos

6. Guía de Instalación

6.1 Clonar el Repositorio

```
git clone https://github.com/emilymontec/MANJARES-CAMPO.git
cd MANJARES-CAMPO
```

6.2 Crear Entorno Virtual e Instalar Dependencias

```
# Crear entorno virtual
python -m venv venv

# Activar en Linux/macOS
source venv/bin/activate

# Activar en Windows
venv\Scripts\activate

# Instalar dependencias
pip install -r requirements.txt
```

6.3 Configurar Variables de Entorno

Crea un archivo .env en la raíz del proyecto con las siguientes variables:

```
# .env
DATABASE_URL=postgres://usuario:password@localhost:5432/manjarescampo_db
WHATSAPP_NUMBER=numero_whatsapp
DEFAULT_SHIPPING_COST=10000
FREE_SHIPPING_OVER=100000
DISCOUNT_PERCENT=0
BUSINESS_HOURS=horarios_habiles
SOCIAL_FACEBOOK_URL=tu_pagina_facebook
SOCIAL_INSTAGRAM_URL=tu_perfil_instagram
```

6.4 Configurar la Base de Datos PostgreSQL

```
# En PostgreSQL (psql o pgAdmin)
CREATE DATABASE manjares_db;
CREATE USER postgres WITH PASSWORD 'tu_contraseña';
GRANT ALL PRIVILEGES ON DATABASE manjares_db TO postgres;
```

6.5 Aplicar Migraciones y Crear Superusuario

```
# Aplicar migraciones
python manage.py makemigrations
python manage.py migrate

# Crear superusuario para el panel admin
python manage.py createsuperuser

# Recopilar archivos estáticos
python manage.py collectstatic
```

6.6 Ejecutar el Servidor de Desarrollo

```
python manage.py runserver

# La aplicación estará disponible en:
# http://127.0.0.1:8000/         → Tienda
# http://127.0.0.1:8000/admin/   → Panel de administración
```

7. Rutas Principales (URLs)

Método	URL	Descripción
GET	/	Página principal/catálogo de productos
GET	/?cat=num_cat/	Listado completo de productos con filtros
GET/POST	/carrito/	Ver y gestionar el carrito de compras
POST	/carrito/	Agregar producto al carrito
POST	/carrito/	Eliminar producto del carrito
GET/POST	Directamente a Whatsapp	Formulario y creación de nuevo pedido
GET	/rastreo/	Detalle y estado de un pedido
GET	/rastreo/?order_id=num_ped	Rastreo del envío de un pedido
GET/POST	/dashboard/admin/	Panel de administración

8. Consideraciones de Seguridad

8.1 Buenas Prácticas Implementadas

- **Variables de entorno**: Claves secretas y credenciales almacenadas en .env, nunca en el código fuente ni en el repositorio.
- **CSRF Protection**: Django incluye protección CSRF activa por defecto en todos los formularios POST.
- **Autenticación del Admin**: El panel de administración requiere usuario y contraseña; solo accesible para usuario administrador.
- **Validación de formularios**: Todos los datos de entrada del usuario son validados mediante los Forms de Django antes de procesar.
- **DEBUG en producción**: La variable DEBUG debe establecerse en False al desplegar en producción.

9. Glosario

Término	Definición
Django	Framework web de Python de alto nivel que fomenta el desarrollo rápido y limpio.
PostgreSQL	Sistema gestor de bases de datos relacional de código abierto.
ORM	Object-Relational Mapping. Técnica de abstracción que mapea objetos Python a tablas SQL.
MVT	Model-View-Template. Patrón de arquitectura de Django.
WSGI	Web Server Gateway Interface. Protocolo estándar para servidores Python.
Plugin WhatsApp	Integración con la API de WhatsApp para envío de mensajes automáticos.
CSRF	Cross-Site Request Forgery. Ataque que Django previene con tokens de seguridad.
Carrito de Compras	Almacenamiento temporal de productos seleccionados antes del pedido final.
Rastreo de Envío	Seguimiento del estado y ubicación de un paquete durante su entrega.

10. Información del Proyecto

- **Repositorio**: https://github.com/emilymontec/MANJARES-CAMPO
- **Autor/es**: Gemima Cerpa, Melany Tesillo, Jailiss Gómez, Emily Monterrosa
- **Licencia**: No aplica licencias
- **Versión de Documentación**: 1.0 — 2026